

BUỔI 2

Hệ thống layout: Flexbox và Grid

Thời lượng: 5 tiết · **Yêu cầu:** đã đạt toàn bộ checklist buổi 1 · **Nội:** commit + tag `buoi-2`

1. MỤC TIÊU

- Chọn đúng Flexbox hay Grid cho từng dạng bố cục, giải thích được lựa chọn.
- Dịch Auto Layout của Figma sang class Tailwind một cách chính xác và có hệ thống.
- Giữ nhịp khoảng cách thống nhất trên toàn trang.

2. ĐẦU VÀO VÀ ĐẦU RA

	Nội dung
Đầu vào	Repo sau buổi 1: navbar và hero đã xong, khung semantic đã đủ
Đầu ra	<code>index.html</code> hoàn chỉnh 10 section ở màn hình từ 1280px trở lên, có nội dung thật

Việc đầu tiên khi vào buổi (10 phút): mở lại code buổi 1, đọc soát và tự sửa ít nhất một lỗi mình phát hiện được.

3. KẾ HOẠCH 5 TIẾT

Tiết	Nội dung	Sản phẩm cuối tiết
1	Quy tắc chọn Flex/Grid, chuẩn hóa container	Bảng dự đoán class cho 6 frame + container chuẩn
2	Dải logo khách hàng + lưới tính năng	Hai section xong
3	Cảm nhận + khối số liệu	Hai section xong
4	Bảng giá ba thẻ cao bằng nhau	Section khó nhất của buổi
5	FAQ, CTA, footer + rà nhịp toàn trang	Commit + tag <code>buoi-2</code>

4. NHIỆM VỤ CHI TIẾT

Nhiệm vụ 1 — Quy tắc chọn Flex hay Grid (tiết 1)

Bài tập khởi động (20 phút). Mở file Figma, tự chọn 6 frame có bố cục khác nhau. Với mỗi frame, viết ra giấy class Tailwind dự đoán **trước khi** mở code, sau đó dựng thử để kiểm chứng dự đoán của mình.

Chỉ cần một câu hỏi để chọn: **các phần tử có cần thẳng hàng với nhau theo cả hai chiều không?**

Tình huống	Chọn	Ví dụ trong bài
Một hàng, kích thước tùy nội dung	<code>flex</code>	Navbar, hàng nút, nội dung bên trong thẻ
Lưới đều nhau, cân thẳng hàng cả dọc lẫn ngang	<code>grid</code>	Thẻ tính năng, bảng giá, cột footer
Danh sách dài ngắn khác nhau, tự xuống dòng	<code>flex flex-wrap</code>	Dải tên khách hàng

Chuẩn hóa container dùng chung cho mọi section: `mx-auto w-full max-w-6xl px-5`.

Quy tắc về khoảng cách — bắt buộc tuân thủ cả buổi: luôn dùng `gap`, không bao giờ đặt `margin` lên từng phần tử con. `gap` chỉ chèn khoảng cách ở giữa, nên phần tử đầu và cuối không thừa lề — thứ mà `margin` luôn làm sai và phải đi vá bằng `:last-child`.

Nhiệm vụ 2 — Dải khách hàng và lưới tính năng (tiết 2)

Dải logo dùng flex tự xuống dòng:

```
<ul class="flex flex-wrap items-center justify-center gap-x-12 gap-y-6">
```

Lưới tính năng, viết luôn cả ba mốc responsive từ đầu:

```
<ul class="mt-14 grid gap-6 sm:grid-cols-2 lg:grid-cols-3">
```

Mỗi thẻ gồm icon trong ô vuông nền nhạt, tiêu đề, mô tả:

```
<span class="grid h-11 w-11 place-items-center rounded-lg bg-brand-50 text-brand-600">
  <svg class="h-6 w-6" ... aria-hidden="true">...</svg>
</span>
```

Hai điểm phải nhớ:

- `grid place-items-center` là cách ngắn nhất để căn giữa cả hai chiều. Dùng lại được cho nút tròn, badge, avatar.
- `aria-hidden="true"` trên SVG trang trí là **bắt buộc**. Icon này không mang thông tin; để trình đọc màn hình đọc nó là gây nhiễu.

Nhiệm vụ 3 — Cảm nhận và số liệu (tiết 3)

Cảm nhận phải dùng đúng thẻ ngữ nghĩa, không phải `<div>` lỏng `<p>`:

```
<figure>
  <blockquote>Trước đây tôi nào tôi cũng ngồi cộng sổ tới 9 giờ...</blockquote>
  <figcaption>
    <span>Tên người</span>
    <cite>Chức danh, đơn vị</cite>
  </figcaption>
</figure>
```

Khối số liệu dùng `<dl>` kèm một kỹ thuật đáng học:

```
<div class="text-center">
  <dt class="order-2 mt-2 text-sm text-muted">Vừa đang dùng hàng ngày</dt>
  <dd class="font-display text-4xl font-extrabold text-brand-600">340</dd>
</div>
```

Trong HTML, `<dt>` (nhãn) phải đứng trước `<dd>` (giá trị) mới đúng ngữ nghĩa, nhưng nhìn thì phải thấy con số to trước. `order-2` đảo **thứ tự hiển thị** mà không đụng tới thứ tự trong mã. Đây là ví dụ chuẩn của việc tách cấu trúc khỏi trình bày.

Dùng `divide-x` để kẻ vạch ngăn giữa các cột thay vì tự thêm border cho từng phần tử.

Nhiệm vụ 4 — Ba thẻ giá cao bằng nhau (tiết 4)

Đây là điểm tắc lớn nhất của buổi. Công thức bốn phần:

```
<ul class="grid items-stretch gap-6 lg:grid-cols-3">      <!-- 1. ô kéo cao bằng nhau -->
  <li class="card flex h-full flex-col">                    <!-- 2. thẻ cao hết ô, xếp dọc -->
    <h3>Tên gói</h3>
    <p>Giá</p>
    <ul>...danh sách tính năng...</ul>
    <a class="btn btn-primary mt-auto">Chọn gói này</a> <!-- 3. mt-auto đẩy nút xuống đáy -->
  </li>
</ul>
```

`mt-auto` là mấu chốt: trong một flex container dọc, `margin-top: auto` nuốt hết khoảng trống còn thừa và đẩy phần tử xuống đáy. Nhờ vậy ba nút thẳng hàng dù danh sách tính năng dài ngắn khác nhau.

Thẻ nổi bật dùng `ring-2 ring-brand-600` và một badge:

```
<li class="card relative flex h-full flex-col ring-2 ring-brand-600">
  <span class="badge absolute -top-3 left-6 bg-brand-600 text-white">Được chọn nhiều nhất</span>
```

Quy tắc về `absolute`: chỉ dành cho phù hiệu, huy hiệu, chấm thông báo — những thứ *nằm đè lên* bố cục. Dùng `absolute` để xếp bố cục chính là sai và sẽ vỡ ngay khi đổi kích thước màn hình. Trong toàn bộ đồ án, `absolute` không được xuất hiện quá 3 lần.

Nhiệm vụ 5 — FAQ, CTA, footer và rà nhịp (tiết 5)

FAQ dùng lưới hai cột không đều — đây là lúc dùng cú pháp cột tùy chỉnh:

```
<div class="grid gap-12 lg:grid-cols-[1fr_1.4fr]">
```

Footer bốn cột: logo và mô tả ngắn, nhóm liên kết sản phẩm, nhóm liên kết hỗ trợ, địa chỉ. Mỗi nhóm liên kết bọc trong `<nav aria-labelledby="...">` với một heading riêng.

Rà nhịp cuối buổi. Kiểm tra mọi `py-` giữa các section phải nằm trong tập { `py-20` , `py-24` , `py-28` }. Chỉ cần một section lệch là cả trang mất nhịp, dù không ai chỉ ra được chính xác chỗ nào sai.

5. YÊU CẦU HOÀN THÀNH

- ☐ `index.html` đủ 10 section, có nội dung thật, khớp Figma ở màn hình từ 1280px
- ☐ Không dùng `position: absolute` để xếp bố cục chính
- ☐ Không có giá trị spacing tùy ý; mọi khoảng cách nằm trong scale
- ☐ Khoảng cách giữa các phần tử dùng `gap`, không dùng `margin` trên từng con
- ☐ Ba thẻ bảng giá cao bằng nhau, ba nút thẳng hàng khi nội dung dài ngắn khác nhau
- ☐ Cảm nhận dùng `<figure>` / `<blockquote>` / `<cite>`; số liệu dùng `<dl>`
- ☐ Mọi SVG trang trí có `aria-hidden="true"`
- ☐ Ít nhất 4 commit, có tag `buoi-2`

6. BÀI TẬP VỀ NHÀ

Thêm **một section không có trong file Figma** vào `index.html`. Gợi ý: "Cách hoạt động" gồm 3 bước, hoặc bảng so sánh với cách làm thủ công hiện tại.

Yêu cầu: section mới phải nhất quán với hệ token và nhịp spacing đang dùng — người xem không được nhận ra đây là phần thêm vào.

Bài này đo khả năng **nội suy một design system**, tức là tạo ra thứ chưa có trong thiết kế nhưng vẫn đúng ngôn ngữ của thiết kế đó. Đây là việc lập trình viên front-end phải làm hằng ngày.

Lưu ý về đánh số. Nếu chọn section "Cách hoạt động" và dùng số thứ tự 01 / 02 / 03, hãy chắc rằng nội dung **thật sự là một chuỗi có thứ tự**. Đánh số cho ba tính năng không liên quan nhau chỉ là trang trí, và sẽ bị trừ điểm ở phần thiết kế.

7. LỖI THƯỜNG GẶP

Hiện tượng	Nguyên nhân	Cách sửa
Ba thẻ giá cao lệch nhau	Thiếu <code>h-full</code> hoặc thiếu <code>items-stretch</code>	Áp lại công thức bốn phần ở nhiệm vụ 4
Nút trong thẻ bị đẩy lung tung	Thiếu <code>mt-auto</code>	Thêm vào phần tử cuối của thẻ flex dọc
Lưới vỡ khi có <code>gap</code>	Dùng <code>w-1/3</code> thay vì <code>grid-cols-3</code>	Chuyển sang grid, để <code>gap</code> tự lo khoảng cách
Phần tử đầu/cuối thừa lề	Dùng <code>margin</code> thay <code>gap</code>	Bỏ hết margin con, đặt <code>gap</code> ở container
Footer xếp chồng kỳ lạ	Dùng <code>float</code>	Chuyển sang <code>grid</code>
Lồng flex nhiều tầng không cần thiết	Chưa phân tích trước khi gõ	Vẽ khung ra giấy trước, mỗi tầng phải có lý do

8. TỰ KIỂM TRA CUỐI BUỔI

Trả lời được cả bốn câu sau (viết ra giấy hoặc ghi vào README) thì coi như đã nắm bài:

1. Khi nào dùng Flexbox, khi nào dùng Grid? Cho ví dụ trong chính bài của bạn.
2. Vì sao `gap` tốt hơn `margin` cho khoảng cách giữa các phần tử?
3. `mt-auto` hoạt động dựa trên nguyên lý nào?
4. Section bạn tự thêm ở bài tập về nhà dùng lại những token nào của thiết kế gốc?